



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Gerência de Configuração

Práticas de Gerência de Configuração

Profa. Lana Mesquita



Conteúdo

- Controle de Versão
- Gerência de Mudanças
- Integração Contínua (CI)
- Deployment Contínuo (CD)
 - Entrega Contínua (*Continuous Delivery*)

Áreas



**Controle de
Versão**



**Controle de
Mudanças**



**Integração
Contínua**



**Deployment
Contínuo**



Controle de Versão



Controle de Versão

O controle de versão é um sistema que **registra as mudanças** feitas em um arquivo ou um conjunto de arquivos ao longo do tempo de forma que você possa **recuperar versões específicas**.



Controle de Versão

É a espinha dorsal de toda a gerência de configuração, apoiando as atividades de controle de mudança e integração contínua. Fornece os seguintes serviços:

- **Identificação**, armazenamento e gerenciamento dos itens de configuração e de suas versões durante todo o ciclo de vida do software;
- **Histórico** de todas as alterações efetuadas nos itens de configuração;
- Criação de **rótulos** e **ramificações** no projeto;
- **Recuperação** de uma configuração em um determinado momento desejado do tempo.



Controle de Versão

Um Sistema de Controle de Versões (VCS, na sigla em inglês) oferece:

- um **repositório** para armazenar a versão mais recente do código fonte de um sistema e de arquivos de documentação, configuração, páginas Web, etc.
 - Fundamental para que os desenvolvedores possam **colaborar** e para que os operadores saibam precisamente **qual versão do sistema deve ser colocada em produção**.
- manter o **histórico das versões** mais importantes de cada arquivo
- permite que se **recupere versões** mais antigas de qualquer arquivo, caso seja necessário (ctrl + z).



Controle de Versão

Termos

- **Repositório:** Local onde se armazenam as configurações.
- **Baseline:** Versão estável, controlada, sobre a qual se desenvolvem novas.
- **Check-out:** Requisição de modificações sobre algo que está num repositório.
- **Check-in:** Atualização do repositório para incorporar modificações feitas no processo de check-out.
- **Versão:** Instância de um dado artefato em um ponto no tempo, diferente de outras versões.



Controle de Versão

Termos

- **Ramos** (Branches): Versões que seguem linhas de desenvolvimento paralelas à principal.
- **Item**: Um artefato gerenciado pelo processo de GC.
- **Construção** (Building): Processo de compilar o sistema a partir de itens fonte
- **Release**: Versão entregue ao cliente.



excalidraw / excalidraw

Q Type / to search



<> Code Issues 1.9k Pull requests 479 Discussions Actions Projects 1 Security 2 Insights

Commits

master

All users

All time

Commits on Aug 29, 2025

fix: bound text rotation across alignments (#9914)

Verified

ae89608



cTangonan123 and A-Mundanilkunathil authored 3 days ago · 7 / 7

Commits on Aug 22, 2025

fix: tighten distance for double tap text creation (#9889)

Verified

3085f4a



ryan-di authored last week · 12 / 12

fix: restore from invalid fixedSegments & type-safer point updates (#9899)

Verified

531f3e5



dwele and mtolmacs authored last week · 7 / 7

fix: calling toLowerCase on potentially undefined navigator.* values (#9901)

Verified

90ec273



dwele authored last week · 7 / 7

Commits on Aug 21, 2025

chore: bump mermaid-to-excalidraw to 1.1.3 (#9898)

Verified

f29e9df



dwele authored last week · 12 / 12

fix: even deltas with version & version nonce are valid (#9897)

Verified

b5ad7ae



Mrazator authored last week · 7 / 7

<https://github.com/excalidraw/excalidraw/commits/master/>



Controle de Versão

Sistemas de
controle de
versão

Lida com arquivos inteiros

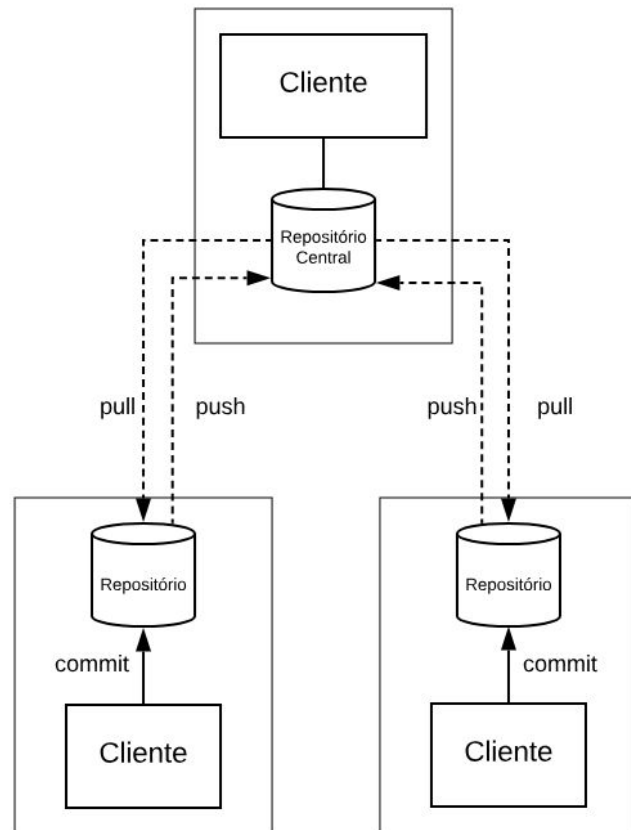
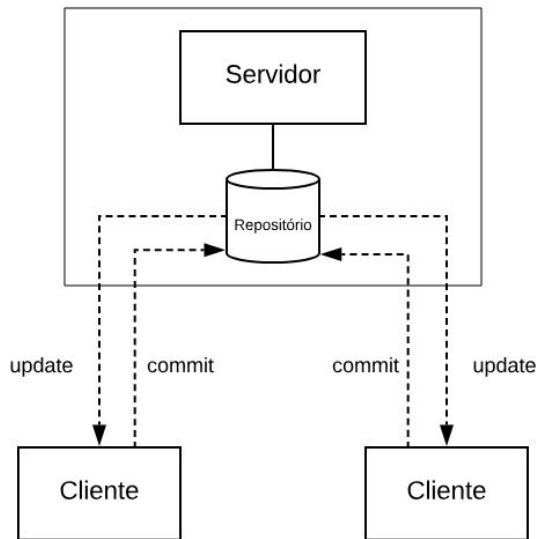


obsoleto



Controle de Versão

Sistemas de controle de versão





Controle de Versão

Plataformas de
armazenamento



GitLab

Git x Github

Git x Github



Git é um sistema de controle de versões distribuído cujo desenvolvimento foi liderado por Linus Torvalds, também responsável pela criação do sistema operacional Linux.



GitHub é um serviço de hospedagem de código que usa o sistema Git para prover controle de versões. O GitHub oferece repositórios públicos e gratuitos, para projetos de código aberto, e repositórios fechados e pagos, para uso por empresas.



Gerência de Mudanças



Gerência de Mudanças

O controle de mudança é uma das atividades necessárias da Gerência de Configuração.

Seu papel principal é **registrar, avaliar e rastrear todas as mudanças aplicadas ao projeto**, desde o momento em que são propostas, até o momento em que são implementadas (ou não) nos itens de configuração.

Mudanças aprovadas são implementadas nos itens de configuração e nos dados e documentos relacionados



Gerência de Mudanças

Acompanhamento do ciclo de vida do pedido de mudança.

***PROPOSTA -> AVALIAÇÕES -> ACEITAÇÃO ->
IMPLEMENTAÇÃO***



Gerência de Mudanças

- Rastreamento da mudança: É importante existir uma **amarração bi-direcional entre a mudança publicada no repositório e o pedido que a gerou.**
- Funcionalidade dependente da **integração com controle de versão.**



Gerência de Mudanças

excalidraw / excalidraw

Search: Type to search

<> Code Issues 1.9k Pull requests 479 Discussions Actions Projects 1 Security 2 Insights

fix: Scene deleted after pica image resize failure #9879

Merged dwelle merged 1 commit into master from zsviczian/fix-scene-dumped-on-image-fail 2 weeks ago

Conversation 7 Commits 1 Checks 12 Files changed 1

+1 -1

zsviczian commented 2 weeks ago Collaborator

fixes #9878

 **Revert change in private updateImageCache** Verified ✓ 87cb1d3

vercel bot commented 2 weeks ago • edited

The latest updates on your projects. Learn more about [Vercel for GitHub](#).

Project	Deployment	Preview	Updated (UTC)
excalidraw	Ready	Preview	Aug 18, 2025 8:42am
excalidraw-package-example	Ready	Preview	Aug 18, 2025 8:42am
excalidraw-package-example-with-nextjs	Ready	Preview	Aug 18, 2025 8:42am

Reviewers

- Mrazator**
- dwelle** ✓

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

<https://github.com/excalidraw/excalidraw/pull/9879>



Gerência de Mudanças

Funcionalidades
de Controle de
Mudança

- **Configuração do fluxo de trabalho**
 - Os estados pelos quais um pedido passa durante o seu ciclo de vida.
 - Depende do processo de desenvolvimento utilizado no projeto. É desejável que a ferramenta possibilite configurar o fluxo de trabalho de acordo com a necessidade de cada projeto.



Gerência de Mudanças

Funcionalidades
de Controle de
Mudança

- **Relatórios Personalizados**
 - As ferramentas costumam oferecer relatórios específicos. Entretanto, às vezes, é necessário criar e disponibilizar determinado relatório para uma necessidade específica.
- **Notificações** para acompanhamento da evolução do pedido
 - Manter informado todos os envolvidos com um pedido de mudança sobre alterações recebidas durante seu ciclo de vida.



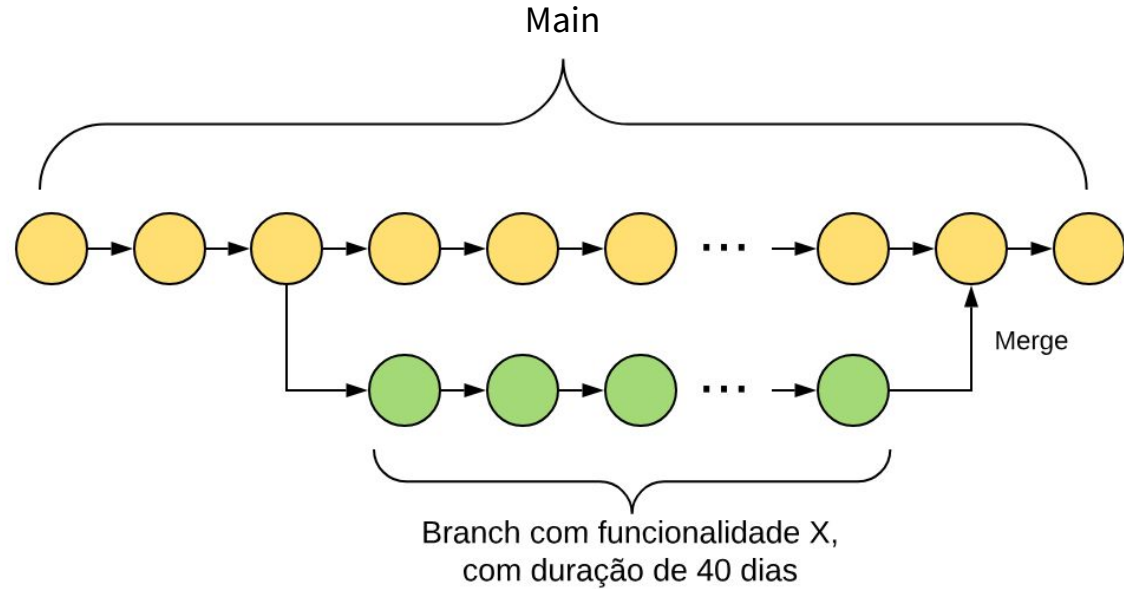
<https://www.menti.com/almtf7a954mj>



Integração Contínua (CI)

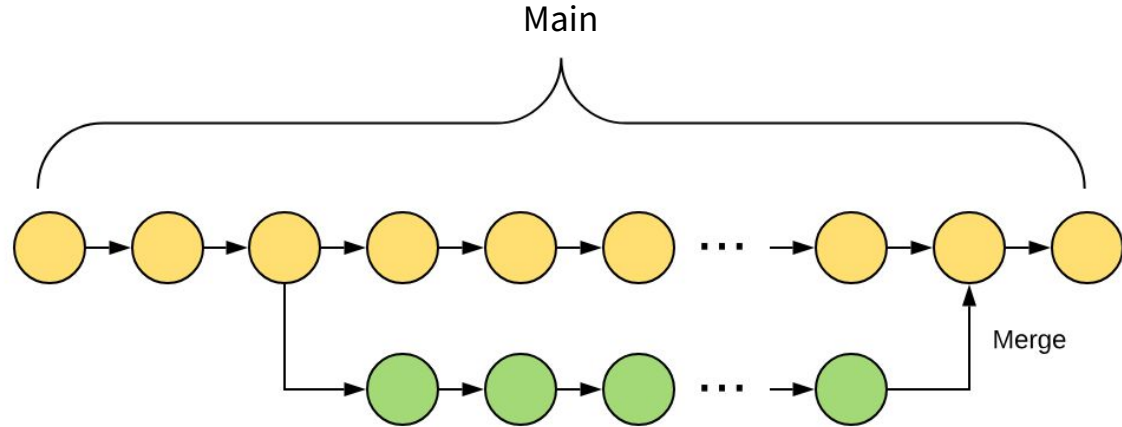


Integração Contínua





Integração Contínua



```

V.test(n||"")||se.error("unsupported lang: "+n,n=n.re
t;do{if(t=E>e.lang;e.getAttribute("xml:lang")||e.getA
nction(e){var t=n.location&&n.location.hash;return t&&
e==C.activeElement&&(C.hasFocus||C.hasFocus())&&!(e
t-e.nodeName.toLowerCase());return"input"===t&&n.e.ch

```

INTEGRATION HELL

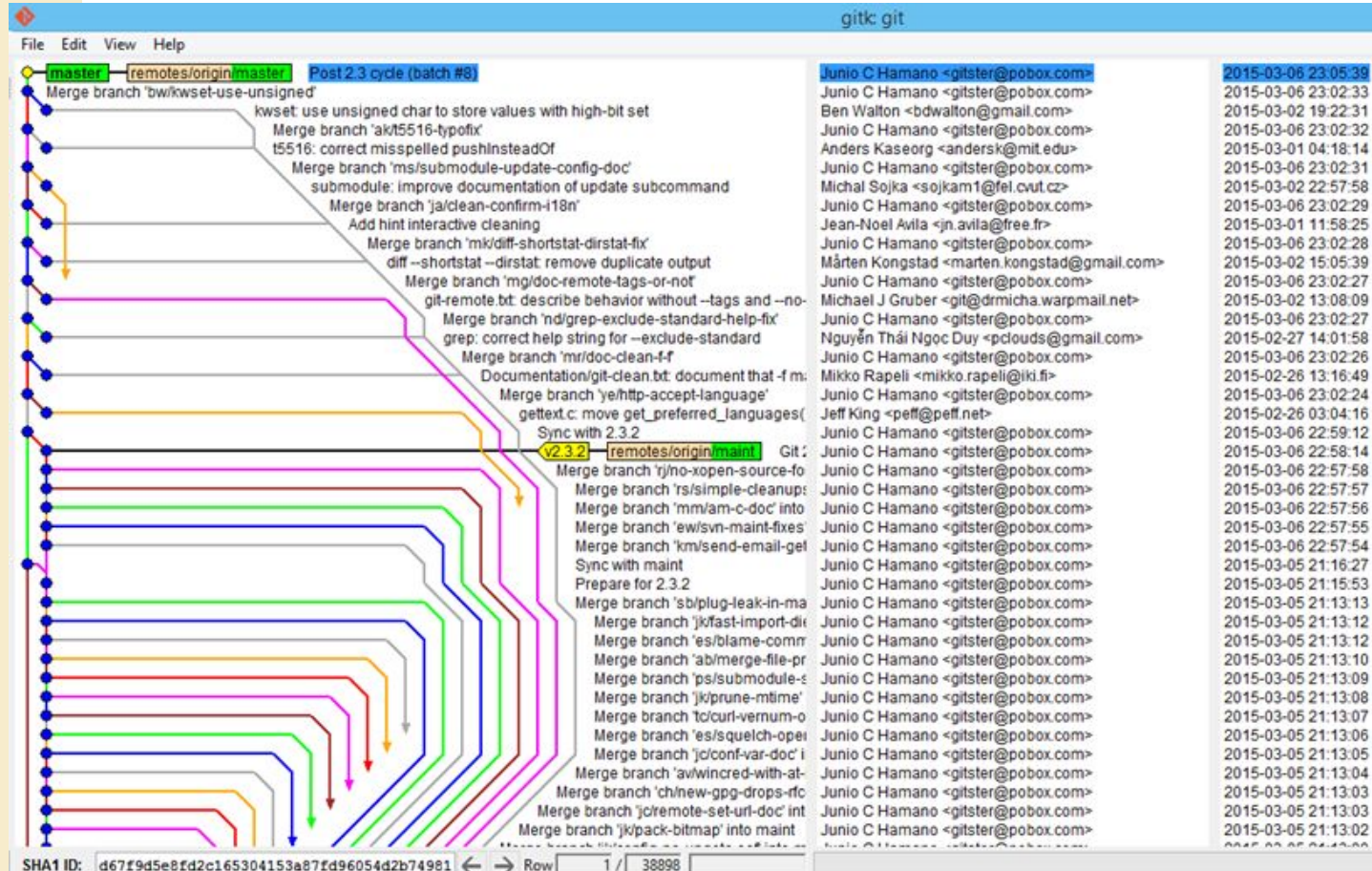
```

s.empty(e),header...on(e);return J.test(e.nodeName
n(t),eq,ve...e,e,t,n);return[n,n-t,n]);ev
e)),lt:ve(function(e,t,n){for(var r=n<0?n+t:t&n?t:n;0
e)}}).pseudos.nth=b.pseudos.eq,{radio:10,checkbox:10,
main}{for(var i=0,n=e.length,r="";t<n;r+=e[t].valu
e.first?function(e,t,n){while(e=e[u])if(1===e.nodeType
r,i,o,a=[k,p];if(n){while(e=e[u])if((1===e.nodeType|t

```



Integração Contínua





Integração Contínua

O objetivo é garantir que **as mudanças no projeto são construídas, testadas e relatadas** o mais rápido possível.

Para isso usa de etapas **automatizadas** para fornecer alterações de maneira confiável.



Você deve integrar e testar o seu código em intervalos menores do que algumas horas. Programação em times não é um problema do tipo dividir-e-conquistar. Na verdade, é um problema que requer dividir, conquistar e integrar. A duração de uma tarefa de integração é imprevisível e pode facilmente levar mais tempo do que a tarefa original de codificação. Assim, quanto mais tempo você demorar para integrar, maiores e mais imprevisíveis serão os custos.

Kent Beck, em seu livro de XP



Integração Contínua

Integração Contínua (Continuous Integration ou CI) é uma prática de desenvolvimento proposta por Extreme Programming (XP).

A proposta do Kent Beck não é consenso. Martin Fowler mencionam pelo menos **uma integração por dia por desenvolvedor**.

<https://agilemanifesto.org/>



Integração Contínua

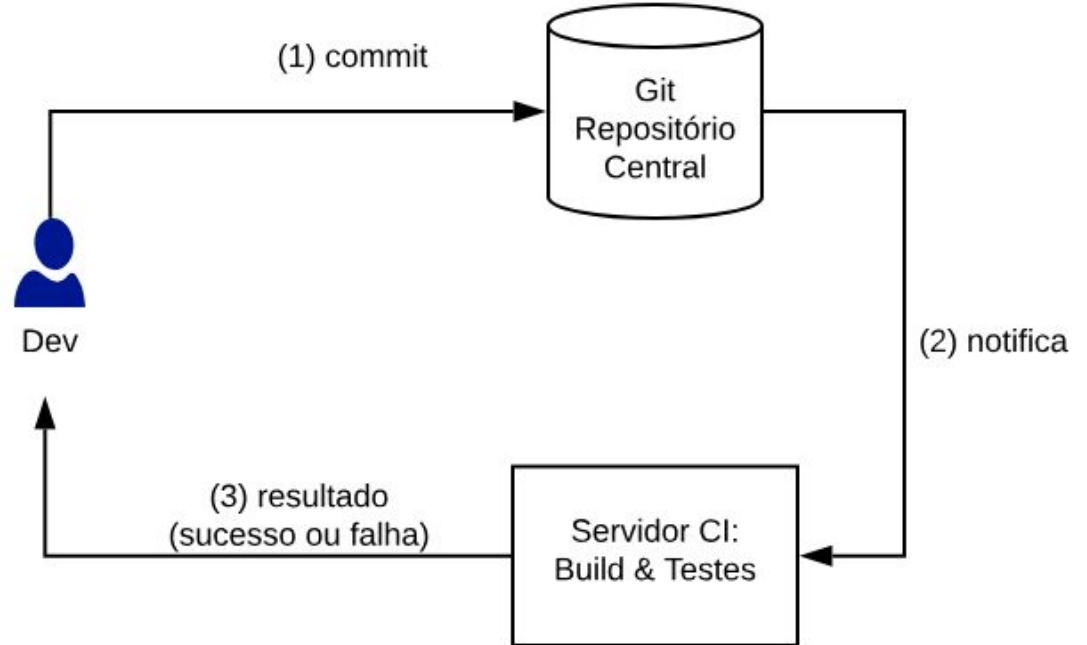
Boas práticas:

- **Build Automatizado** (< 10mn)
 - Build é a compilação de todos os arquivos de um sistema, até a geração de uma versão executável
- **Testes Automatizados**
 - garantir que o sistema compila sem erros após cada novo commit.
- **Servidores de Integração Contínua:** ambientes para executar os testes automatizados (ex. Github Actions)



Integração Contínua

▪ Servidores de Integração Contínua



**Se o servidor de CI notificar
o desenvolvedor de que seu
código não passou nos
testes ou quebrou o build,
ele deve parar tudo o que
está fazendo e providenciar
a correção.**



Integração Contínua

CI é compatível com o uso de branches? Depende.

“Na maioria das vezes, branches de funcionalidades constituem uma abordagem incompatível com CI. Um dos princípios de CI é que todos devem enviar commits para a linha de desenvolvimento principal diariamente. Então, a não ser que os branches de funcionalidades durem menos do que um dia, eles são um animal diferente de CI. É comum ouvir desenvolvedores dizendo que eles estão usando CI porque eles rodam builds automáticos, talvez usando um servidor de CI, após cada commit. Isso pode ser chamado de building contínuo e pode ser uma coisa boa. Porém, como não há integração, não podemos chamar essa prática de CI.” (Martin Fowler)



Integração Contínua

Benefícios:

- Melhorar qualidade do código
- Reduzir tempo para entregar atualizações
- Promover colaboração entre os membros da equipe
- Garantir que a base do código esteja sempre funcional
- Permite ciclos de feedback mais rápidos
- Permite corrigir identificar e corrigir problemas rapidamente
- Redução de riscos



Integração Contínua

Quando não usar

Quando não usar CI?

A regra de **pelo menos uma integração por dia** por desenvolvedor depende da organização, do domínio do sistema (que pode ser um sistema crítico) e do perfil dos desenvolvedores (que podem ser iniciantes). Talvez seja mais factível realizar uma integração a cada dois ou três dias.

***Práticas de Engenharia de Software — incluindo CI —
não devem ser consideradas ao pé da letra, isto é,
exatamente como está descrita nos livros.***



Integração Contínua

Quando não usar

Quando não usar CI?

CI também não é compatível com **projetos de código livre**. Na maioria das vezes, os desenvolvedores desses projetos são voluntários e não têm disponibilidade para trabalhar diariamente no seu código.

Nesses casos, um modelo baseado em Pull Requests e Forks, conforme usado pelo GitHub, é mais adequado.



Integração Contínua

Ferramentas





Deployment Contínuo



Deployment Contínuo

Com CI o código novo é frequentemente integrado no branch principal, mesmo que não esteja pronto para entrar em produção.

Quando usa-se CD, **todo novo commit que chega na *main* entra rapidamente em produção.**



Deployment Contínuo

1. O desenvolvedor **desenvolve e testa** na sua máquina local.
2. Ele realiza um **commit** e o servidor de CI executa novamente um **build** e os **testes de unidade**.
3. Algumas vezes no dia, o servidor de CI realiza **testes** mais exaustivos com os novos commits que ainda não entraram em produção. Esses testes incluem, por exemplo, testes de integração, testes de interface e testes de desempenho.
4. **Se todos os testes passarem, os commits entram imediatamente em produção.** E os usuários já vão interagir com a nova versão do código.



Deployment Contínuo

Vantagens

- CD reduz o tempo de entrega de novas funcionalidades.
- CD torna novas releases (ou implantações) um não-evento. Não existe mais um dia D ou um deadline para entrega de novas releases.
- Feedback mais rápido de usuários reais do que foi entregue.
- CD favorece experimentação e um estilo de desenvolvimento orientado por dados e feedback dos usuários.

**Em cada atualização, na média,
92 linhas de código são
adicionadas ou modificadas.
Esses números revelam que,
para funcionar bem, CD requer
que as atualizações de código
sejam pequenas.**

Sobre uma equipe no Facebook



Deployment Contínuo

Entrega Contínua
(Continuous
Delivery)

Deployment Contínuo (CD) não é recomendável para certos tipos de sistemas, incluindo sistemas desktop (como uma IDE ou um navegador Web), aplicações móveis e aplicações embutidas em hardware.

**Já pensou ter que atualizar o app do
Whatsapp todo dia? 😐**



Deployment Contínuo

Entrega Contínua
(Continuous
Delivery)

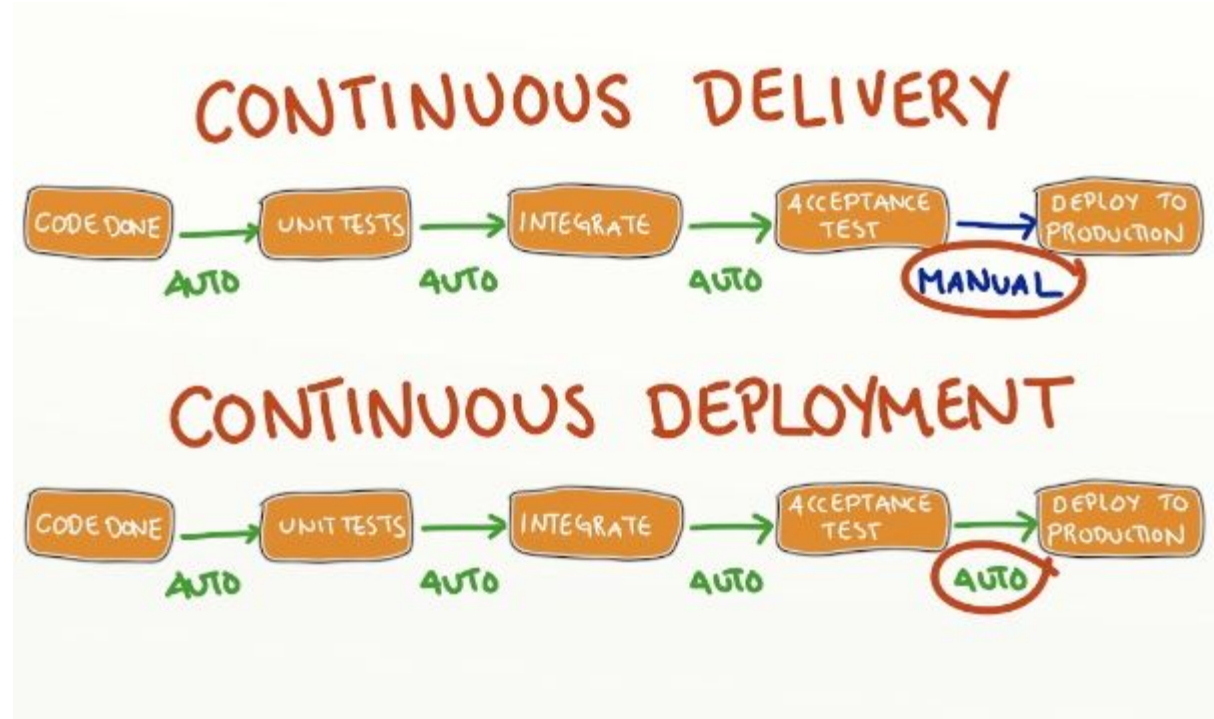
Nestes sistemas pode-se usar um versão mais fraca de CD, chamada de **Entrega Contínua (Continuous Delivery)**.

- **Delivery** é o processo de liberar uma nova versão de um sistema para ser objeto de deployment.
- **Deployment** é o processo de liberar uma nova versão de um sistema para seus usuários.



Deployment Contínuo

Entrega Contínua
(Continuous
Delivery)





Deployment Contínuo

Feature Flags

Se novas releases são liberadas quase todo dia, como evitar que minhas implementações parciais, que ainda não foram devidamente testadas ou que têm problemas de desempenho, cheguem até os usuários finais?

Feature Flags

Integre continuamente o código parcial da funcionalidade X, mas com ela desabilitada, isto é, qualquer código relativo a X estará guardado por uma variável booleana (um flag).



Deployment Contínuo

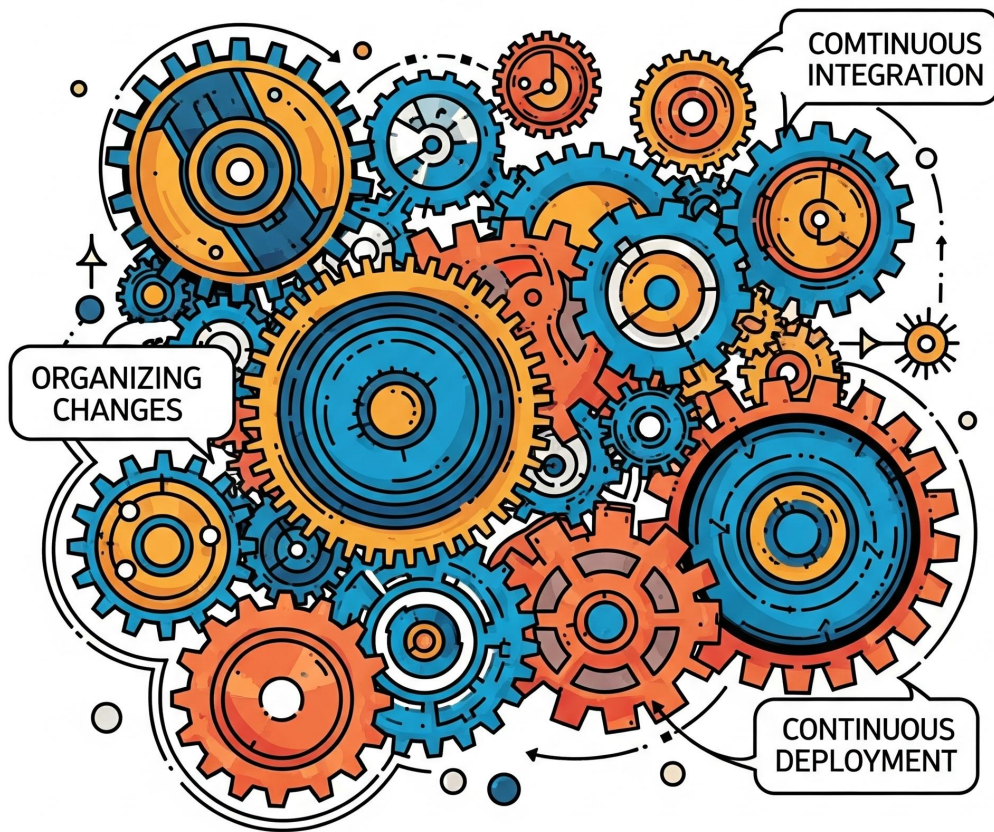
Feature Flags

```
nova_pag = false;  
...  
if (nova_pag)  
    "carregue nova página"  
else  
    "carregue página antiga"
```




<https://www.menti.com/alzvojpokvs5>

Software Configuration Management



Revisão Geral

- **Controle de Versão** registra e recupera versões do código.
- **Gerência de Mudanças** rastreia alterações do projeto e pode ser feita no Github ou outras ferramentas.
- **Integração Contínua** automatiza “subir” o código e incentiva integrar menos código e mais rapidamente.
- **Deployment Contínuo e Delivery Contínuo** agilizam as entregas ao cliente, o Delivery tem a última etapa manual para garantir que não entre em produção algo que não é esperado pelos outros stakeholders.

Dúvidas?
Manda no Discord!



Até a próxima aula!

Lana Mesquita

